



Sistema de Recomendação em Grafos com PLN

Como recomendar livros de forma inteligente, unindo similaridade de conteúdo textual e comportamento de leitores — usando grafos dirigidos, TF-IDF e algoritmos clássicos de ciência da computação.

ESTRUTURAS DE DADOS 2 — FGA0030

UNB · 2026.1

PROF. GLAUCO / LENON

O Problema: Sobrecarga de Informação

Com milhares de livros publicados anualmente, escolher a próxima leitura tornou-se um desafio. Sistemas tradicionais de recomendação dependem de metadados manuais ou de grandes volumes de dados históricos — recursos nem sempre disponíveis em contextos acadêmicos ou projetos independentes.

O Desafio Central

Como recomendar livros de forma inteligente, combinando **similaridade semântica de texto** (via PLN) com o **comportamento de outros leitores**?

A Nossa Solução

Um sistema de recomendação baseado em **grafos dirigidos ponderados**, que extrai semântica das resenhas via TF-IDF e cruza dados de usuários com métricas de grafos clássicas.



Modelagem do Grafo

O sistema utiliza um **digrafo ponderado gerado dinamicamente**, com três tipos de vértices e dois tipos de arestas que codificam tanto o comportamento dos usuários quanto a semântica dos textos.

Vértices — 3 Tipos

USUÁRIO

Leitores cadastrados no sistema

TEXTO

Livros contendo resenhas textuais

GÊNERO

Categorias literárias com palavras-chave

Arestas — 2 Tipos

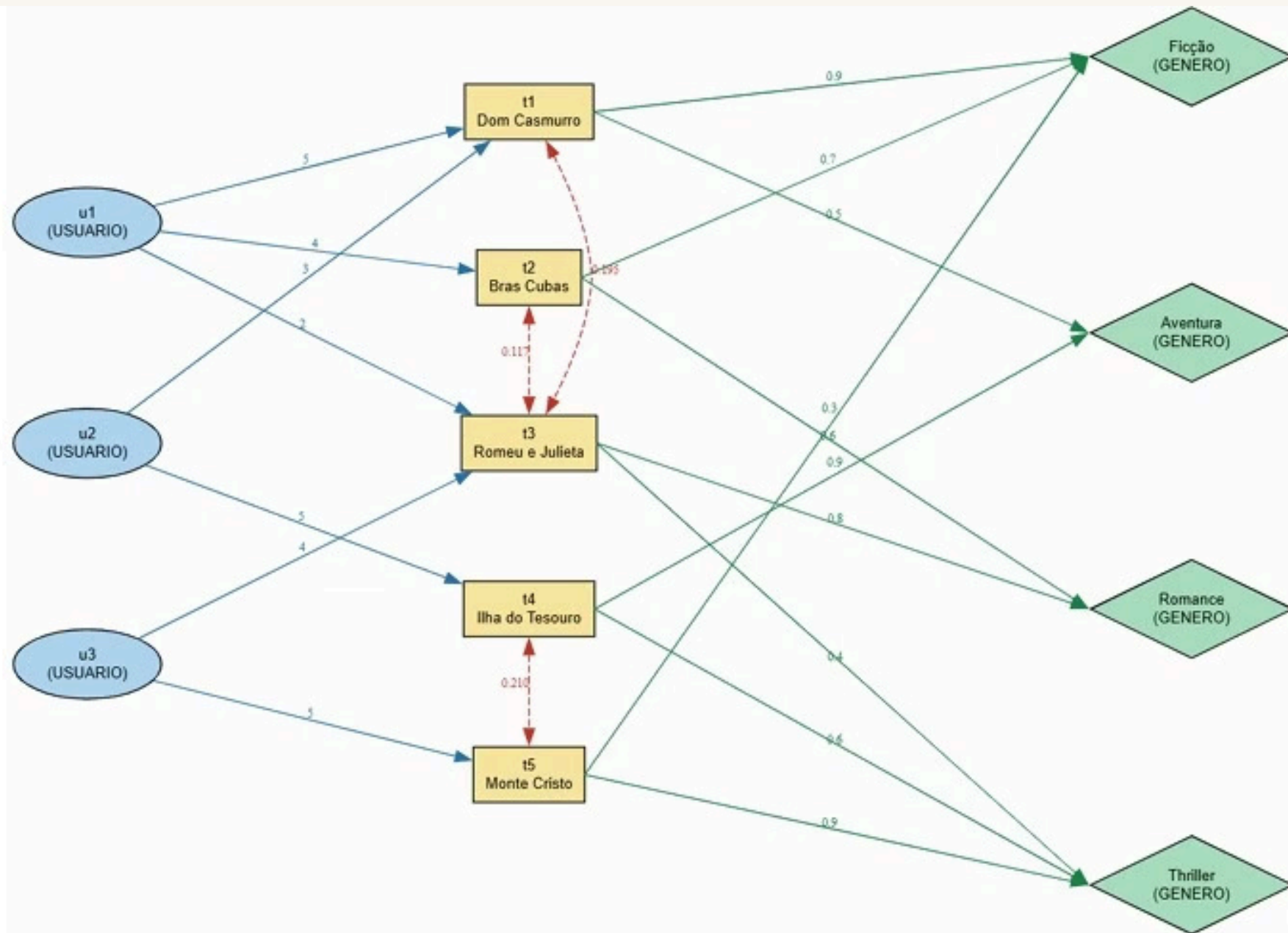
→ Aresta Normal

Usuário → Texto: Peso = Nota (1 a 5)

Texto → Gênero: Peso = Cosseno TF-IDF

→ Projeção Semântica

Texto ↔ Texto: Bidirecional, peso igual à similaridade cosseno entre as resenhas dos livros



Implementação e Estruturas de Dados

O sistema foi desenvolvido **100% em C (C99)**, otimizado com a flag `-O2` e gerenciado via CMake. As estruturas de dados foram escolhidas para garantir eficiência em cada etapa do pipeline.



Grafo — Listas de Adjacência

Implementação nativa do digrafo ponderado, permitindo iteração eficiente sobre vizinhos e inserção dinâmica de arestas durante o processamento PLN.



Tabela Hash — $O(1)$ Amortizado

Desenvolvida do zero com função de espalhamento **djb2** e tratamento de colisões por **sondagem linear**. Usada para indexar o vocabulário de PLN com acesso instantâneo.



Filas (Queues)

Estrutura essencial para o processamento do **algoritmo BFS** (busca em largura) e do **algoritmo de Kahn** (ordenação topológica), ambos centrais no pipeline de recomendação.

O Pipeline de 5 Fases

A solução é organizada em um pipeline sequencial de cinco fases, cada uma responsável por extrair um tipo diferente de sinal de recomendação. O resultado final é um ranking ponderado que combina semântica, comportamento e afinidade de gênero.

Cada fase contribui com uma dimensão distinta para o score final: a **Fase 0** extrai semântica pura das resenhas; as **Fases 1 e 2** exploram a estrutura do grafo de usuários e livros; a **Fase 3** ranqueia gêneros por afinidade; e a **Fase 4** incorpora o filtro colaborativo via similaridade entre usuários.

Fase 0 — PLN: Tokenização, Hash e TF-IDF

A fase de Processamento de Linguagem Natural é a fundação do sistema. Sem intervenção humana, o grafo cria arestas dinâmicas entre livros e gêneros com base puramente no conteúdo textual das resenhas.

01

Tokenização

As resenhas são normalizadas (ASCII, sem acentos) e quebradas em tokens. Pontuação e stopwords são removidas para formar o vocabulário do corpus.

02

Indexação via Hash

Cada token é mapeado para um índice inteiro usando a Tabela Hash com função djb2, garantindo lookup em **$O(1)$ amortizado** durante o cálculo das frequências.

03

Cálculo TF-IDF

Para cada livro, calcula-se o peso de cada termo considerando sua frequência no documento e sua raridade no corpus. Gera vetores esparsos de alta dimensionalidade.

04

Similaridade Cosseno

O cosseno entre vetores TF-IDF cria as arestas **Texto** ↔ **Texto** (projeção semântica) e **Texto** → **Gênero**, conectando livros a categorias temáticas de forma automática.

Fases 1 a 4 — Algoritmos de Grafo

Após a construção do grafo semântico, quatro algoritmos clássicos trabalham em conjunto para filtrar, medir distâncias, ranquear gêneros e identificar comunidades de leitores similares.

1

BFS — Busca de Candidatos

Busca em Largura com **lookup inverso** para encontrar livros não lidos pelo usuário-alvo. Inclui **Fallback Semântico**: caso não haja candidatos diretos, explora vizinhos semânticos via projeção

Texto ↔ Texto.

2

Dijkstra — Distância Personalizada

Algoritmo de Dijkstra com **pesos invertidos**: notas altas (5 estrelas) tornam-se caminhos mais curtos. Livros bem avaliados por usuários similares ficam "mais próximos" no grafo.

3

Kahn — Afinidade de Gênero

Ordenação Topológica (Algoritmo de Kahn) para ranquear os **gêneros favoritos do usuário**. Gêneros com maior fluxo de arestas ponderadas recebem pontuação máxima na recomendação.

4

Jaccard — Filtro Colaborativo

Índice de Jaccard mede a **similaridade entre usuários** com base nas obras em comum. Usuários com Jaccard alto formam uma "bolha" colaborativa que influencia o score final.

Exemplo de Entrada e Saída

Entrada — Dados Sintéticos

Foram cadastrados usuários, notas e resenhas de **23 livros** em ASCII (sem acentos). Exemplo de resenha:

"amor e paixao definem esta historia de forma intensa e marcante"

O sistema exibe no console a **matriz de similaridade cosseno** (texto×texto e texto×gênero) antes de gerar o ranking final.

Saída — Ranking para u1

O sistema exibe o ranking final de recomendação para o usuário-alvo **u1**, detalhando a contribuição de cada uma das 5 fases para cada livro candidato.

Livro t10 — Vencedor

Maior score agregado: Kahn máximo (Romance = gênero favorito), Jaccard 66,6% com a bolha de leitores, e forte ligação TF-IDF com livros já lidos por u1.

Score Detalhado

Cada livro candidato recebe uma nota por fase, permitindo análise de sensibilidade e depuração do impacto de cada algoritmo no resultado final.

Resultados e Escalabilidade

O sistema demonstrou alta eficiência e escalabilidade linear ao ser testado com bases de dados crescentes. O tempo de execução permaneceu estável mesmo com a duplicação do corpus sintético.

23

Livros no Corpus

Resenhas fictícias geradas com coerência temática via LLMs, formatadas em ASCII para compatibilidade com a tokenização em C.

66,6%

Jaccard Máximo

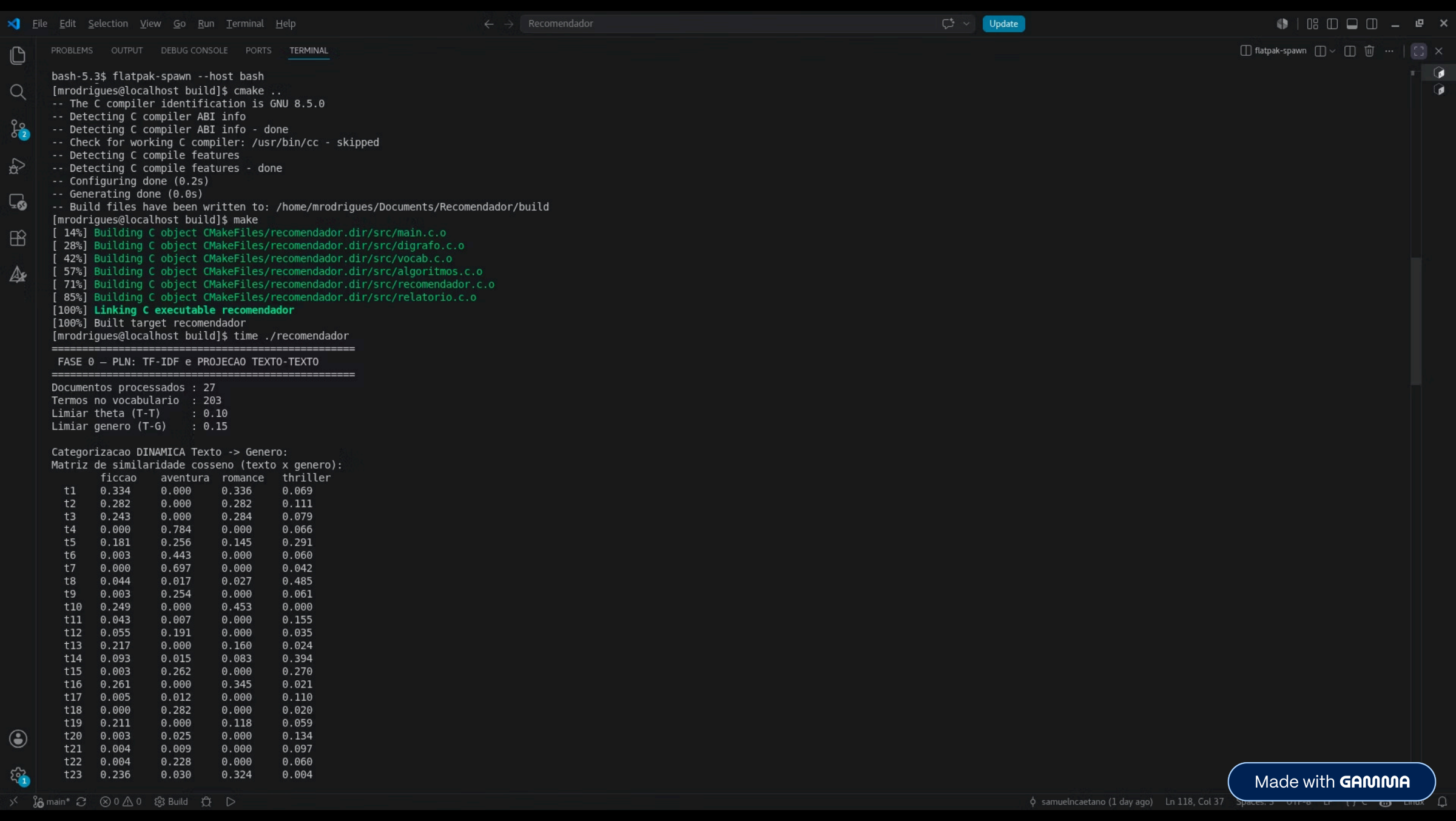
Similaridade entre u_1 e a "bolha" de usuários que leram o livro t_{10} — evidenciando a eficácia do filtro colaborativo.

5ms

Tempo de Execução

`real 0m0,005s` medido via comando `time` no Linux, mesmo após dobrar a base de dados com novos usuários e livros sintéticos.

- ✓ O comportamento **$O(n)$** foi comprovado empiricamente: ao dobrar a base de dados, o tempo de execução permaneceu praticamente inalterado, confirmando a escalabilidade do pipeline.



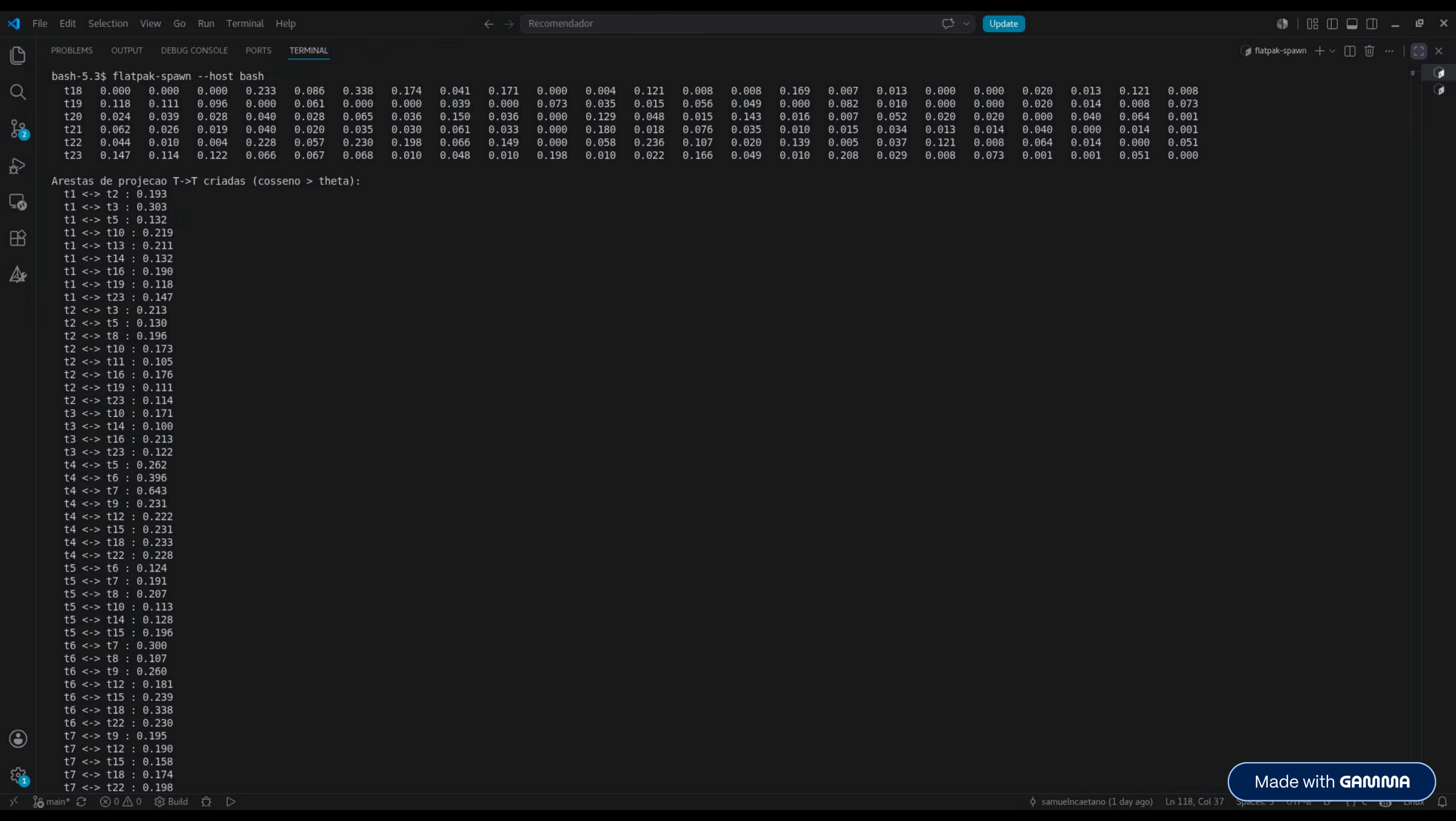
```

bash-5.3$ flatpak-spawn --host bash
t23 0.236 0.030 0.324 0.004

Arestas T->G criadas (coseno > 0.15): 30
t1 -> romance : 0.336
t1 -> ficcao : 0.334
t2 -> romance : 0.282
t2 -> ficcao : 0.282
t3 -> romance : 0.284
t3 -> ficcao : 0.243
t4 -> aventura : 0.784
t5 -> thriller : 0.291
t5 -> aventura : 0.256
t5 -> ficcao : 0.181
t6 -> aventura : 0.443
t7 -> aventura : 0.697
t8 -> thriller : 0.485
t9 -> aventura : 0.254
t10 -> romance : 0.453
t10 -> ficcao : 0.249
t11 -> thriller : 0.155
t12 -> aventura : 0.191
t13 -> romance : 0.160
t13 -> ficcao : 0.217
t14 -> thriller : 0.394
t15 -> thriller : 0.270
t15 -> aventura : 0.262
t16 -> romance : 0.345
t16 -> ficcao : 0.261
t18 -> aventura : 0.282
t19 -> ficcao : 0.211
t22 -> aventura : 0.228
t23 -> romance : 0.324
t23 -> ficcao : 0.236

Matriz de similaridade coseno (texto x texto):
t1 t2 t3 t4 t5 t6 t7 t8 t9 t10 t11 t12 t13 t14 t15 t16 t17 t18 t19 t20 t21 t22 t23
t1 0.000 0.193 0.303 0.019 0.132 0.017 0.015 0.067 0.096 0.219 0.048 0.042 0.211 0.132 0.032 0.190 0.012 0.000 0.118 0.024 0.062 0.044 0.147
t2 0.193 0.000 0.213 0.000 0.130 0.001 0.000 0.196 0.001 0.173 0.105 0.010 0.075 0.062 0.001 0.176 0.069 0.000 0.111 0.039 0.026 0.010 0.114
t3 0.303 0.213 0.000 0.000 0.070 0.001 0.000 0.046 0.073 0.171 0.038 0.016 0.076 0.100 0.000 0.213 0.021 0.000 0.096 0.028 0.019 0.004 0.122
t4 0.019 0.000 0.000 0.000 0.262 0.396 0.643 0.043 0.231 0.078 0.029 0.222 0.076 0.023 0.231 0.050 0.031 0.233 0.000 0.040 0.040 0.228 0.066
t5 0.132 0.130 0.070 0.262 0.000 0.124 0.191 0.207 0.085 0.113 0.053 0.079 0.083 0.128 0.196 0.094 0.018 0.086 0.061 0.028 0.020 0.057 0.067
t6 0.017 0.001 0.001 0.396 0.124 0.000 0.300 0.107 0.260 0.000 0.052 0.181 0.056 0.047 0.239 0.043 0.028 0.338 0.000 0.065 0.035 0.230 0.068
t7 0.015 0.000 0.000 0.643 0.191 0.300 0.000 0.029 0.195 0.031 0.021 0.190 0.018 0.014 0.158 0.009 0.021 0.174 0.000 0.036 0.030 0.198 0.010
t8 0.067 0.196 0.046 0.043 0.207 0.107 0.029 0.000 0.159 0.035 0.231 0.038 0.068 0.276 0.210 0.026 0.223 0.041 0.039 0.150 0.061 0.066 0.048
t9 0.096 0.001 0.073 0.231 0.085 0.260 0.195 0.159 0.000 0.056 0.028 0.204 0.018 0.017 0.255 0.009 0.027 0.171 0.000 0.036 0.033 0.149 0.010
t10 0.219 0.173 0.171 0.078 0.113 0.000 0.031 0.035 0.056 0.000 0.026 0.031 0.088 0.077 0.000 0.311 0.064 0.000 0.073 0.000 0.000 0.198 0.198
t11 0.048 0.105 0.038 0.029 0.053 0.052 0.021 0.231 0.028 0.026 0.000 0.042 0.025 0.172 0.012 0.066 0.102 0.004 0.035 0.129 0.180 0.058 0.010
t12 0.042 0.010 0.016 0.222 0.079 0.181 0.190 0.038 0.204 0.031 0.042 0.000 0.029 0.018 0.134 0.018 0.019 0.121 0.015 0.048 0.018 0.236 0.022
t13 0.211 0.075 0.076 0.076 0.083 0.056 0.018 0.068 0.018 0.088 0.025 0.029 0.000 0.034 0.042 0.213 0.030 0.008 0.056 0.015 0.076 0.107 0.166
t14 0.132 0.062 0.100 0.023 0.128 0.047 0.014 0.276 0.017 0.077 0.172 0.018 0.034 0.000 0.108 0.048 0.108 0.008 0.049 0.143 0.035 0.020 0.049
t15 0.032 0.001 0.000 0.231 0.196 0.239 0.158 0.210 0.255 0.000 0.012 0.134 0.042 0.108 0.000 0.040 0.014 0.169 0.000 0.016 0.010 0.139 0.010
t16 0.190 0.176 0.213 0.050 0.094 0.043 0.009 0.026 0.009 0.311 0.066 0.018 0.213 0.048 0.040 0.000 0.008 0.007 0.082 0.007 0.015 0.005 0.208
t17 0.012 0.069 0.021 0.031 0.018 0.028 0.021 0.223 0.027 0.064 0.102 0.019 0.030 0.108 0.014 0.008 0.000 0.013 0.010 0.052 0.034 0.037 0.029
t18 0.000 0.000 0.000 0.233 0.086 0.338 0.174 0.041 0.171 0.000 0.004 0.121 0.008 0.008 0.169 0.007 0.013 0.000 0.000 0.020 0.013 0.121 0.008
t19 0.118 0.111 0.096 0.000 0.061 0.000 0.000 0.039 0.000 0.073 0.035 0.015 0.056 0.049 0.000 0.087 0.010 0.000 0.000 0.020 0.014 0.008 0.073

```

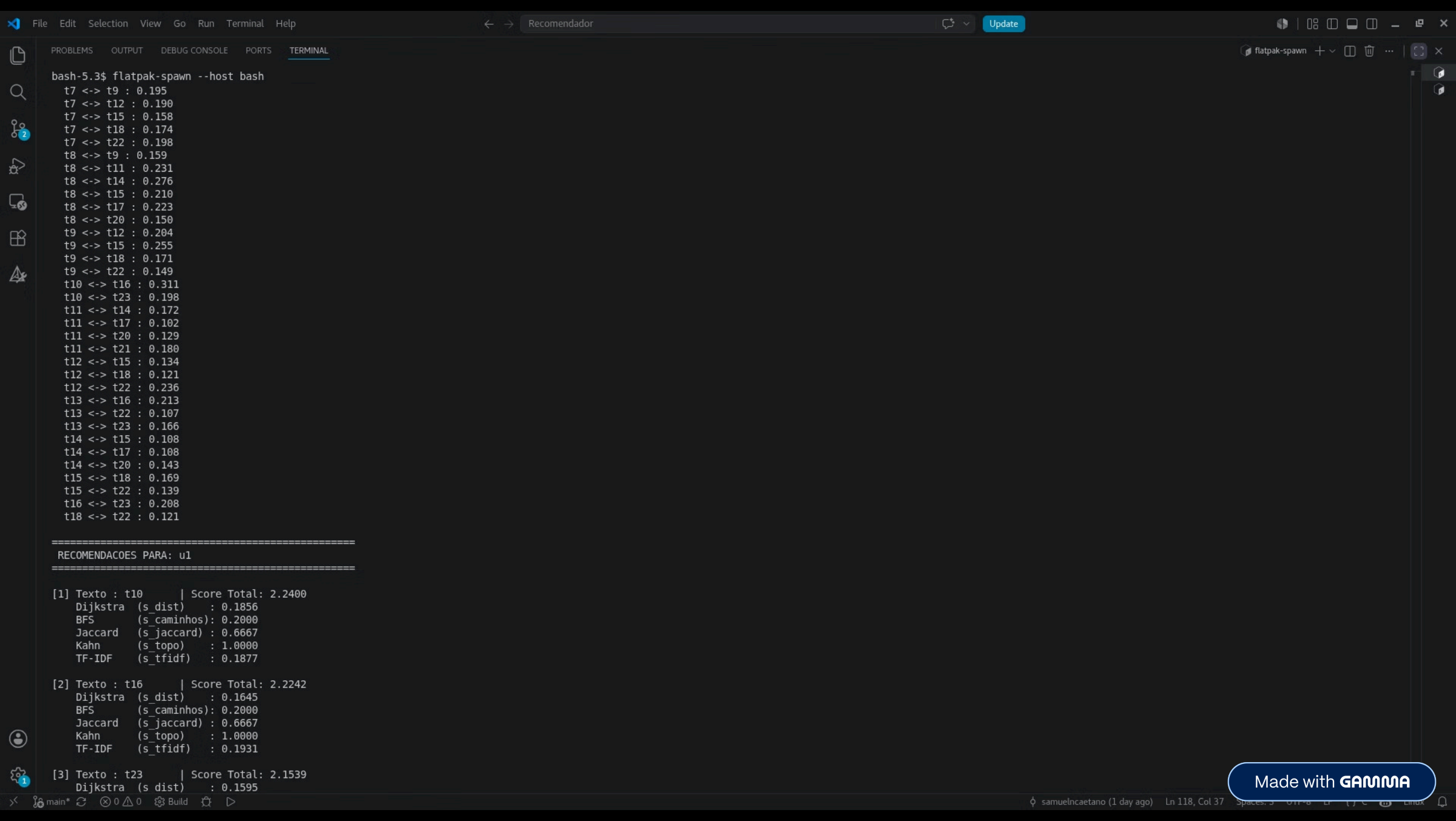


bash-5.3\$ flatpak-spawn --host bash

t18	0.000	0.000	0.000	0.233	0.086	0.338	0.174	0.041	0.171	0.000	0.004	0.121	0.008	0.008	0.169	0.007	0.013	0.000	0.000	0.020	0.013	0.121	0.008
t19	0.118	0.111	0.096	0.000	0.061	0.000	0.000	0.039	0.000	0.073	0.035	0.015	0.056	0.049	0.000	0.082	0.010	0.000	0.000	0.020	0.014	0.008	0.073
t20	0.024	0.039	0.028	0.040	0.028	0.065	0.036	0.150	0.036	0.000	0.129	0.048	0.015	0.143	0.016	0.007	0.052	0.020	0.020	0.000	0.040	0.064	0.001
t21	0.062	0.026	0.019	0.040	0.020	0.035	0.030	0.061	0.033	0.000	0.180	0.018	0.076	0.035	0.010	0.015	0.034	0.013	0.014	0.040	0.000	0.014	0.001
t22	0.044	0.010	0.004	0.228	0.057	0.230	0.198	0.066	0.149	0.000	0.058	0.236	0.107	0.020	0.139	0.005	0.037	0.121	0.008	0.064	0.014	0.000	0.051
t23	0.147	0.114	0.122	0.066	0.067	0.068	0.010	0.048	0.010	0.198	0.010	0.022	0.166	0.049	0.010	0.208	0.029	0.008	0.073	0.001	0.001	0.051	0.000

Arestas de projecao T->T criadas (cosseno > theta):

t1 <-> t2 : 0.193
t1 <-> t3 : 0.303
t1 <-> t5 : 0.132
t1 <-> t10 : 0.219
t1 <-> t13 : 0.211
t1 <-> t14 : 0.132
t1 <-> t16 : 0.190
t1 <-> t19 : 0.118
t1 <-> t23 : 0.147
t2 <-> t3 : 0.213
t2 <-> t5 : 0.130
t2 <-> t8 : 0.196
t2 <-> t10 : 0.173
t2 <-> t11 : 0.105
t2 <-> t16 : 0.176
t2 <-> t19 : 0.111
t2 <-> t23 : 0.114
t3 <-> t10 : 0.171
t3 <-> t14 : 0.100
t3 <-> t16 : 0.213
t3 <-> t23 : 0.122
t4 <-> t5 : 0.262
t4 <-> t6 : 0.396
t4 <-> t7 : 0.643
t4 <-> t9 : 0.231
t4 <-> t12 : 0.222
t4 <-> t15 : 0.231
t4 <-> t18 : 0.233
t4 <-> t22 : 0.228
t5 <-> t6 : 0.124
t5 <-> t7 : 0.191
t5 <-> t8 : 0.207
t5 <-> t10 : 0.113
t5 <-> t14 : 0.128
t5 <-> t15 : 0.196
t6 <-> t7 : 0.300
t6 <-> t8 : 0.107
t6 <-> t9 : 0.260
t6 <-> t12 : 0.181
t6 <-> t15 : 0.239
t6 <-> t18 : 0.338
t6 <-> t22 : 0.230
t7 <-> t9 : 0.195
t7 <-> t12 : 0.190
t7 <-> t15 : 0.158
t7 <-> t18 : 0.174
t7 <-> t22 : 0.198



bash-5.3\$ flatpak-spawn --host bash

```
t7 <-> t9 : 0.195
t7 <-> t12 : 0.190
t7 <-> t15 : 0.158
t7 <-> t18 : 0.174
t7 <-> t22 : 0.198
t8 <-> t9 : 0.159
t8 <-> t11 : 0.231
t8 <-> t14 : 0.276
t8 <-> t15 : 0.210
t8 <-> t17 : 0.223
t8 <-> t20 : 0.150
t9 <-> t12 : 0.204
t9 <-> t15 : 0.255
t9 <-> t18 : 0.171
t9 <-> t22 : 0.149
t10 <-> t16 : 0.311
t10 <-> t23 : 0.198
t11 <-> t14 : 0.172
t11 <-> t17 : 0.102
t11 <-> t20 : 0.129
t11 <-> t21 : 0.180
t12 <-> t15 : 0.134
t12 <-> t18 : 0.121
t12 <-> t22 : 0.236
t13 <-> t16 : 0.213
t13 <-> t22 : 0.107
t13 <-> t23 : 0.166
t14 <-> t15 : 0.108
t14 <-> t17 : 0.108
t14 <-> t20 : 0.143
t15 <-> t18 : 0.169
t15 <-> t22 : 0.139
t16 <-> t23 : 0.208
t18 <-> t22 : 0.121
```

=====

RECOMENDACOES PARA: u1

=====

[1] Texto : t10 | Score Total: 2.2400

Dijkstra	(s_dist)	: 0.1856
BFS	(s_caminhos)	: 0.2000
Jaccard	(s_jaccard)	: 0.6667
Kahn	(s_topo)	: 1.0000
TF-IDF	(s_tfidf)	: 0.1877

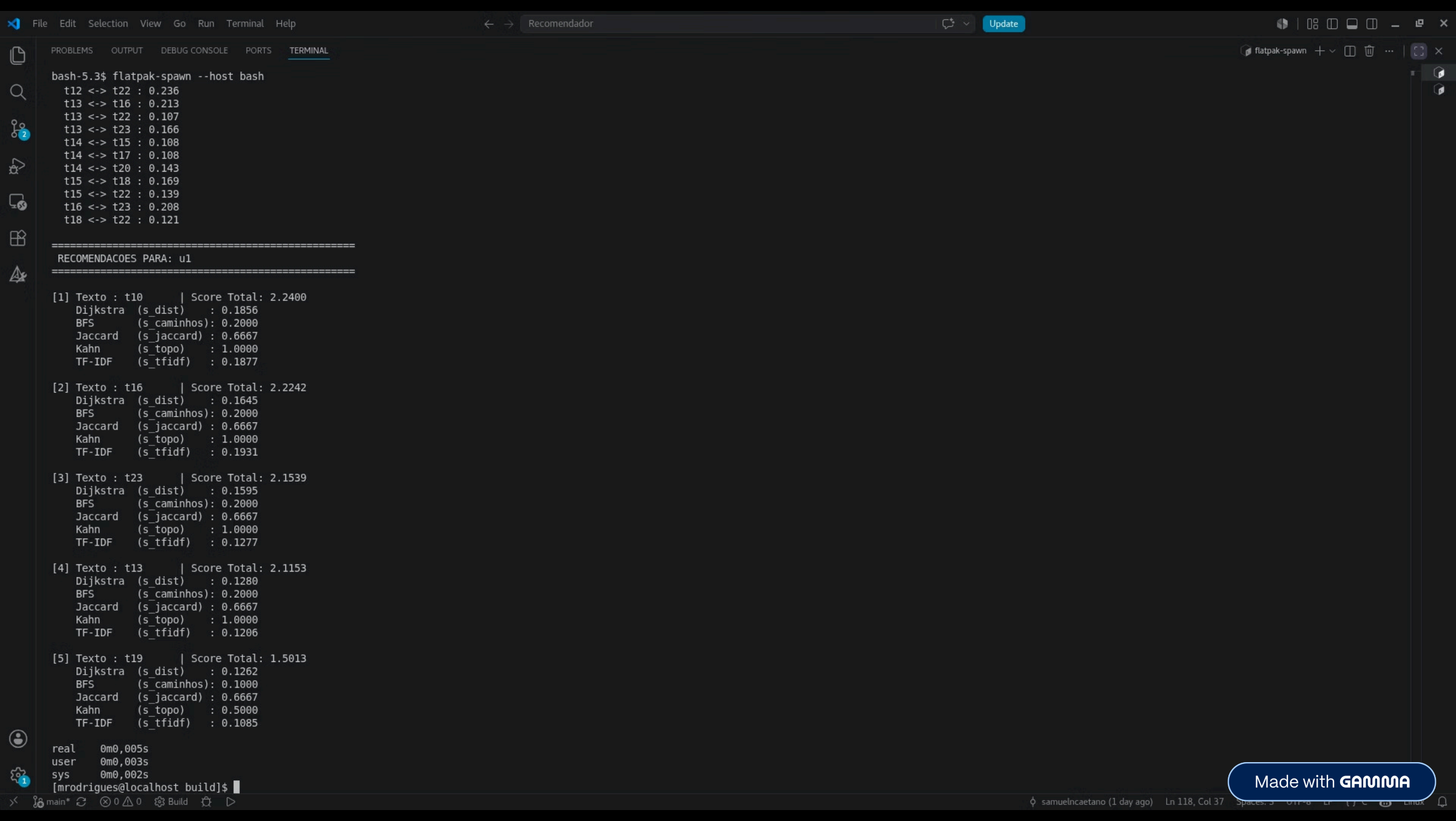
[2] Texto : t16 | Score Total: 2.2242

Dijkstra	(s_dist)	: 0.1645
BFS	(s_caminhos)	: 0.2000
Jaccard	(s_jaccard)	: 0.6667
Kahn	(s_topo)	: 1.0000
TF-IDF	(s_tfidf)	: 0.1931

[3] Texto : t23 | Score Total: 2.1539

Dijkstra	(s_dist)	: 0.1595
----------	----------	----------

Made with GAMMA



```
bash-5.3$ flatpak-spawn --host bash
```

```
t12 <=> t22 : 0.236
t13 <=> t16 : 0.213
t13 <=> t22 : 0.107
t13 <=> t23 : 0.166
t14 <=> t15 : 0.108
t14 <=> t17 : 0.108
t14 <=> t20 : 0.143
t15 <=> t18 : 0.169
t15 <=> t22 : 0.139
t16 <=> t23 : 0.208
t18 <=> t22 : 0.121
```

```
=====
RECOMENDACOES PARA: u1
=====
```

```
[1] Texto : t10 | Score Total: 2.2400
```

```
Dijkstra (s_dist) : 0.1856
BFS (s_caminhos): 0.2000
Jaccard (s_jaccard): 0.6667
Kahn (s_topo) : 1.0000
TF-IDF (s_tfidf) : 0.1877
```

```
[2] Texto : t16 | Score Total: 2.2242
```

```
Dijkstra (s_dist) : 0.1645
BFS (s_caminhos): 0.2000
Jaccard (s_jaccard): 0.6667
Kahn (s_topo) : 1.0000
TF-IDF (s_tfidf) : 0.1931
```

```
[3] Texto : t23 | Score Total: 2.1539
```

```
Dijkstra (s_dist) : 0.1595
BFS (s_caminhos): 0.2000
Jaccard (s_jaccard): 0.6667
Kahn (s_topo) : 1.0000
TF-IDF (s_tfidf) : 0.1277
```

```
[4] Texto : t13 | Score Total: 2.1153
```

```
Dijkstra (s_dist) : 0.1280
BFS (s_caminhos): 0.2000
Jaccard (s_jaccard): 0.6667
Kahn (s_topo) : 1.0000
TF-IDF (s_tfidf) : 0.1206
```

```
[5] Texto : t19 | Score Total: 1.5013
```

```
Dijkstra (s_dist) : 0.1262
BFS (s_caminhos): 0.1000
Jaccard (s_jaccard): 0.6667
Kahn (s_topo) : 0.5000
TF-IDF (s_tfidf) : 0.1085
```

```
real 0m0,005s
```

```
user 0m0,003s
```

```
sys 0m0,002s
```

```
[mrodrigues@localhost build]$
```

Equipe e Contribuições



Samuel

Responsável pela arquitetura e implementação completa do sistema de recomendação em C (C99), incluindo a modelagem do digrafo ponderado (usuários, textos e gêneros), o pipeline de PLN com TF-IDF e similaridade cosseno para categorização dinâmica de textos, e a implementação dos algoritmos BFS, Dijkstra, Kahn e Jaccard utilizados nas fases de descoberta de candidatos, cálculo de distância, afinidade por gênero e similaridade entre usuários. Também foi responsável pela criação do sistema de scoring multi-critério com explicabilidade, pela organização modular do código-fonte (6 módulos em C com headers), pela configuração do build com CMake e pela elaboração da documentação técnica completa do projeto (README).



Thiago

Auxiliou no desenvolvimento do pipeline de PLN e do motor de recomendação (TF-IDF, categorização dinâmica Texto→Gênero e mecanismo de fallback semântico), participando das discussões técnicas.

Expansão de Capacidade: Identificou a necessidade e aumentou inicialmente os limites do grafo (MAX_VERTICES 20→60, MAX_VOCAB 256→512) para viabilizar a integração de dados de todo o grupo, validando que o comportamento do sistema permaneceu inalterado de forma inicial (time, real 0m0,002s).

Expansão de Dados: Adição do usuário u4 e 3 novos livros (resumos sintéticos gerados por IA), validando a categorização automática Texto→Gênero via TF-IDF.



Letícia

Criação dos slides, **testes de estresse de complexidade**, Revisão do código, expansão do banco de dados sintético, Criação dos Slides e Imagens(Claude/Gemini).



Marjorie

Expansão de Dados: Adição do usuário u9 e 3 novos livros com resenhas sintéticas geradas por IA.

Refatoração: Dobrou a capacidade da Tabela Hash (MAX_VOCAB = 1024) para suportar o novo volume no TF-IDF sem falhas.

Escalabilidade final: Comprovou a eficiência da arquitetura medindo o tempo de execução via comando `time` (0m0,005s).



Gabriela

Desenvolvimento em Par (Co-autoria): Atuou ativamente com Letícia e Marjorie na expansão do banco de dados sintético (via IA) e na refatoração crítica da Tabela Hash (MAX_VOCAB = 1024), garantindo que o cálculo de TF-IDF suportasse o novo volume de dados sem falhas.

Validação de Escalabilidade: Apoiou a execução técnica dos testes de estresse de complexidade, auxiliando na validação do tempo de resposta final da arquitetura (time 0m0,005s).

Síntese e Apresentação: Responsável por traduzir a complexidade do código C99 e do pipeline de grafos para os slides de defesa. Realizou a estruturação lógica, design e revisão técnica da apresentação, alinhando o roteiro à documentação oficial.

Uso de Inteligência Artificial (LLMs)

Conforme os critérios da disciplina, a utilização de modelos de linguagem (Claude e Gemini) foi estritamente delimitada a papéis de apoio, garantindo a autoria original do grupo sobre a modelagem dos grafos e a implementação dos 5 algoritmos principais.

A IA foi utilizada para:

- **Geração de Dados Sintéticos (Corpus):** Criação das resenhas textuais fictícias para os 23 livros do escopo. O prompt garantiu que os textos fossem gerados em ASCII e sem acentuação, evitando quebras na função de tokenização nativa em C.
- **Apoio no Refatoramento (Fase 0):** Assistência técnica para adaptar o algoritmo de cálculo do TF-IDF. A IA auxiliou na lógica de vetorizar Textos e Gêneros no mesmo espaço matemático para automatizar a criação dinâmica das arestas `Texto -> Gênero` por similaridade de cosseno.